AI-Assisted Retinal Screening for Cardiovascular Risk

Integrated into a Hospital Management System

Complete project reference: preprocessing, three segmentation models, EfficientNet-B0 classifier, system workflow

> **How to read this document.** Parts marked **[Existing]** describe what is already in the uploaded HospitalManagementSystem project. Parts marked **[Rebuild]** describe the target design agreed in this conversation. No new training has been run yet, so every result for the rebuild is left as a blank to fill in. Numbers quoted for the existing model come from the project's own `metrics.json`; numbers for U-Net, Residual U-Net and Attention U-Net come from your paper draft.

1. Project in one page

The project is a hospital web application that can screen a patient from a single retinal (fundus) photograph. A doctor uploads the photo, the system cleans the image, finds the blood vessels, and then estimates three things: cardiovascular disease (CVD) history, systemic hypertension (high blood pressure), and the grade of diabetic retinopathy (DR). The result, a risk level and a recommendation are stored against the patient in the hospital database and shown in a report.

The idea rests on one medical fact: the retina is the only place where small blood vessels can be seen directly and without surgery. Narrowed arterioles, unusual branching and microvascular damage in the retina often mirror what is happening in the heart and kidneys. Deep learning can pick up these patterns, but only if the vessels are visible, which is why preprocessing and vessel segmentation are central to the design.

The project has four layers:

- **Hospital layer** (Java, Servlets, JSP, MySQL): patients, doctors, appointments, billing, reports, and the CVD screening pages.
- **Preprocessing layer** (Python): green-channel extraction followed by CLAHE contrast enhancement, at a larger image size.
- **Segmentation layer** (PyTorch): three vessel-segmentation models, U-Net, Residual U-Net and Attention U-Net, selectable by the user at prediction time.
- **Classification layer** (PyTorch): an EfficientNet-B0 multi-task classifier that takes the image plus the vessel mask and predicts CVD, hypertension and DR grade.

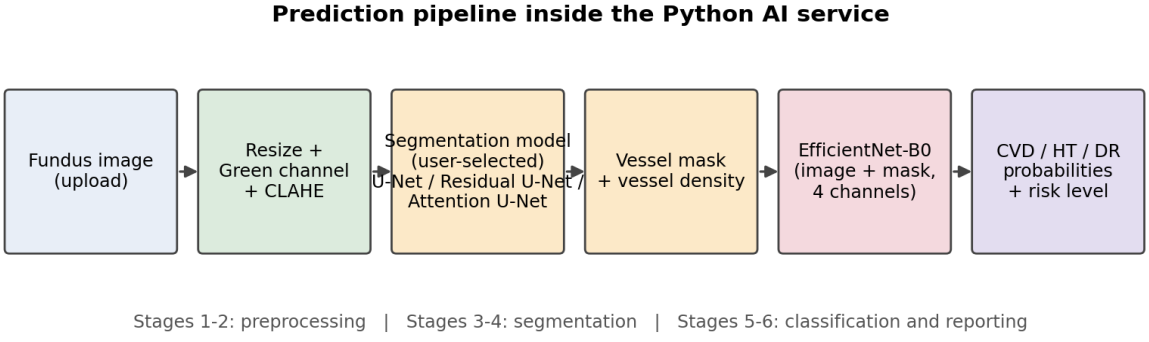


Figure 1. Prediction pipeline inside the Python AI service [Rebuild]

2. Why the retina, and what is being predicted

Fundus photograph. A colour photo of the back of the eye showing the retina, optic disc, macula and blood vessels. It is non-invasive and quick, and handheld smartphone-based cameras make it usable in small clinics.

Target	What it means	Type of output	Why the retina carries a signal
Cardiovascular disease (CVD)	History of vascular disease or myocardial infarction (as labelled in the dataset)	Binary: probability and Positive/Negative	Retinal vessel calibre, tortuosity and density reflect systemic vascular health. The signal is weak and hard to learn.
Hypertension (HT)	History of high blood pressure	Binary	High pressure narrows arterioles and changes the arteriole-to-venule ratio.
Diabetic retinopathy (DR)	Damage to retinal vessels from diabetes, ICDR grade 0 to 4 (No DR, Mild, Moderate, Severe, Proliferative)	5-class probabilities	Microaneurysms, haemorrhages and new vessels are directly visible.

> **Important limit.** The system does **not** predict diabetes. Almost every patient in the training data already has diabetes, so there are no non-diabetic controls. DR grade is a retinal finding, not a diabetes diagnosis. The whole system is a research prototype, not a certified medical device.

3. What exists today [Existing]

3.1 Hospital application

Part	Technology / files	Role
Web tier	Jakarta Servlets and JSP: `CVDServlet`, `PatientServlet`, `DoctorServlet`, `AppointmentServlet`, `BillingServlet`, `LoginServlet`, `ReportServlet`	Handle requests, forward to JSP pages
Data access	DAO classes (`CVDScreeningDAO`, `PatientDAO`, ...) with `DBConnection` and the MySQL connector JAR	All SQL lives here, using PreparedStatements
Model classes	`CVDScreening`, `Patient`, `Doctor`, `Appointment`, `Bill`, `User`	Plain data objects
AI client	`CVDClient` (Java `HttpClient`)	Checks `/health`, posts the image as multipart form data to `/predict`, parses the JSON with regex
Pages	`new-screening.jsp`, `cvd-screenings.jsp`, `cvd-report.jsp` plus the other hospital pages	Upload form, screening list, per-patient report
Storage	MySQL table `cvd_screenings`	Patient id, doctor id, image path, CVD / HT / DR results, risk level, notes

3.2 Existing AI service

A FastAPI service in `cvd-ai-service/` loads one network, `MultiTaskNet`: an EfficientNet-B0 backbone (ImageNet normalisation, 224 by 224 input) with three linear heads on the 1280-dimensional pooled feature. A dropout of 0.4 is applied before the heads. Endpoints are `/` , `/health`, `/predict`, `/predict-base64` and `/sample-images`. A separate Gradio app (`app.py`) offers a standalone demo.

Head	Output	Activation	Threshold
`head_cvd`	1 logit	Sigmoid	0.3238 (Youden's J on validation)
`head_ht`	1 logit	Sigmoid	0.5165 (Youden's J on validation)
`head_dr`	5 logits	Softmax	Argmax; flagged low confidence if top probability is below 0.50 or margin to second class below 0.15

The training loss in the README is a weighted sum of the three tasks: **L = BCE(HT) + 2.0 x BCE(CVD) + CE(DR)**. CVD is weighted double because it is the main target and the hardest. Training used patient-level stratified splits so that images of one patient never appear in both training and test sets.

`compute_overall_assessment` turns the raw outputs into a risk level: **High Risk** if CVD is Positive with probability 0.60 or more; **Moderate Risk** if CVD is Positive, or HT is Positive with probability 0.55 or more, or DR is Moderate or worse; otherwise **Low Risk**.

3.3 Measured performance of the existing model

From `evaluation/metrics.json` (patient-level held-out test set, 191 patients, 593 images, mBRSET):

Task	Key results	Reading
Hypertension	ROC-AUC 0.458 (95% CI 0.363 to 0.550); sensitivity 0.615; specificity 0.357	No better than chance
CVD	ROC-AUC 0.563 (CI 0.471 to 0.662); sensitivity 0.952; specificity 0.054	Near chance. The threshold flags almost everyone: 141 of 149 healthy patients are false positives
DR (5 grades)	Accuracy 0.735; weighted F1 0.656; macro F1 0.188	Mostly predicts the majority class, No DR

> **Inconsistency to be aware of.** The README quotes better numbers (HT AUC 0.672, CVD AUC 0.556, DR accuracy 78.4%) on a larger test set (328 patients). `metrics.json` is the more recent evaluation file, so treat the README table as outdated. Separately, your own combined BRSET + mBRSET notebook reached a **validation** ROC-AUC of 0.8412 for CVD over 60 epochs; a validation score is optimistic, so the held-out test score is the number to defend.

3.4 Known weaknesses that motivate the rebuild

- Only one model, trained on one dataset (mBRSET), at a small 224 by 224 resolution that blurs thin vessels.
- No vessel segmentation and no vessel-focused preprocessing, although the paper's whole argument is that these improve vessel visibility.
- Image-only CVD prediction is near chance; the classifier gets no extra help about where the vessels are.
- No training script in the project, a hard-coded Windows path in `/sample-images`, CORS open to all origins, and no `model_name` stored with each screening.

4. Requirements from the document and how the design meets them

The document is your paper draft, *Retinal Vessel Segmentation Through Green Channel Enhancement and Comparative Deep Learning Architectures*. It sets the requirements the rebuilt project has to satisfy.

Requirement in the paper	How the project meets it [Rebuild]
Three preprocessing options compared: baseline, CLAHE, green channel + CLAHE, using entropy, CII and EBCM	`preprocessing/preproc_metrics.py` recomputes the table at the new image size; green + CLAHE is the production choice
Four vessel datasets: DRIVE (20 images), STARE (40), CHASE_DB1 (28), HRF (45), total 133	Pooled into `data/vessel/`, split by image
Patch-based learning with vessel-containing patches	Overlapping vessel patches, enlarged from 256 to 512 pixels; tiled full-image inference for deployment
Three architectures under one framework: U-Net, Residual U-Net, Attention U-Net	All three live in `segmentation/models/`, trained by one script with identical settings
Dice loss, Adam, learning rate 1e-4, batch size 8	Kept as the baseline recipe; batch size may drop at 512 px
Metrics: Dice, IoU, accuracy, sensitivity, specificity, AUC; loss curves; ROC curves	`evaluate_seg.py` produces the tables and figures
Conclusion points to extracting systemic-disease markers from fundus images	The vessel mask feeds the EfficientNet-B0 classifier, linking segmentation to CVD, HT and DR prediction

Two deliberate changes from the paper: training runs for **100 epochs** instead of 25, and images are processed at **512 pixels** instead of 256. Because of these changes your final tables will not match the paper's numbers exactly, and you should present them as a new experiment.

5. Datasets

Dataset	Used for	Content	Labels
DRIVE	Segmentation	20 images	Manual vessel masks
STARE	Segmentation	40 images	Manual vessel masks
CHASE_DB1	Segmentation	28 images (children)	Manual vessel masks
HRF	Segmentation	45 images, high resolution	Manual vessel masks
mBRSET	Classification	Handheld-camera images of diabetic patients (Phelcom Eyer), 1,289 patients with images on disk	CVD, hypertension, DR grade
BRSET	Classification (second dataset)	Brazilian ophthalmological images, desktop camera	Clinical metadata including hypertension, DR

The two groups never mix: vessel datasets have **masks but no disease labels**; BRSET and mBRSET have **disease labels but no masks**. That is why the pipeline has a bridging step: the best segmentation model is run over BRSET and mBRSET to **generate masks**, and those generated masks become the extra input for the classifier.

Why patient-level and image-level splitting matters. Images from the same patient or the same eye look alike. If some go to training and others to testing, the test score is inflated because the model has effectively seen the answer. The paper splits vessel patches by source image; the classifier splits by patient.

6. Preprocessing

6.1 Why the green channel

A colour image has red, green and blue channels. Blood absorbs green light strongly, so vessels appear darkest against the background in the green channel. The red channel is bright and

washed out, the blue channel is noisy and dark. Using only the green channel keeps the signal and drops the noise.

6.2 Why CLAHE

Contrast Limited Adaptive Histogram Equalisation improves contrast locally. The image is divided into small tiles; each tile gets its own histogram equalisation, with a clip limit that stops noise from being over-amplified. Plain histogram equalisation works on the whole image and cannot cope with uneven lighting, which is common in fundus photographs. Applied directly to RGB, CLAHE also amplifies background noise, which is why the paper applies it after green-channel extraction.

6.3 Evaluation metrics for preprocessing

Metric	Formula (idea)	Higher means
Entropy	H = -sum of p(i) log2 p(i), over intensity levels i	More information content and detail preserved
Contrast Improvement Index (CII)	CII = C(processed) / C(original)	A larger contrast gain over the original
Edge-Based Contrast Measure (EBCM)	Average gradient magnitude along detected edge pixels	Sharper, more distinct vessel borders

Results from the paper (Table I):

Method	Entropy	CII	EBCM
Baseline (RGB)	2.82	0.245	7.61
CLAHE only	3.55	0.233	22.42
Green + CLAHE	3.65	0.264	24.09

Green + CLAHE wins on all three measures, so it is the preprocessing used in production. The fundus image is also resized to the working resolution (512 pixels in the rebuild) before these steps.

7. The three segmentation models

Image segmentation assigns every pixel a label. Here the label is binary: vessel or background. The output is a **mask** with the same size as the input. All three models follow the same **encoder-decoder** shape: the encoder repeatedly halves the image size while increasing the number of feature channels (learning *what* is in the image); the decoder doubles the size back up (learning *where*); and **skip connections** copy encoder features across to the decoder so that fine detail lost during downsampling is recovered. The three models differ only in how each block is built and what happens on the skip path.

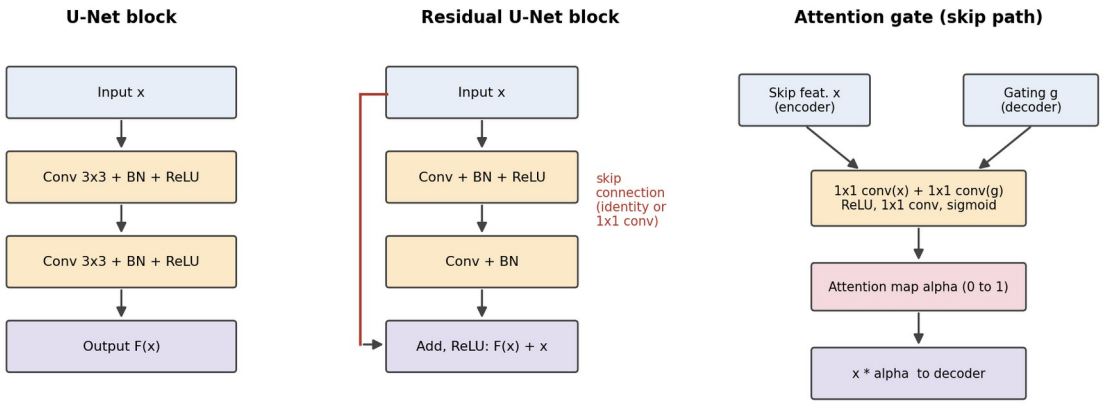


Figure 2. The building block that distinguishes each model [Rebuild]

7.1 U-Net (baseline)

Each stage has two 3 by 3 convolutions with batch normalisation and ReLU. Skip connections concatenate encoder features with decoder features. It is simple, fast and the standard baseline for biomedical segmentation. Its weakness is that very thin vessels are easily lost, and plain stacked convolutions are harder to train as depth grows.

7.2 Residual U-Net (best in the paper, and the one being modified)

Each block adds a **residual (shortcut) connection**: the output is F(x) + x, where F is the two-convolution path. When the channel counts differ, the shortcut uses a 1 by 1 convolution. This helps because gradients flow straight through the shortcut, which makes deep networks train faster and more stably, and each block only has to learn a small correction to its input rather than a whole new representation. In the paper this gave the best Dice, IoU, sensitivity and AUC, and the fastest loss convergence.

Planned modifications to your current Residual U-Net:

- Train on 512 by 512 patches instead of 256 by 256, so thin vessels cover more pixels.
- Train for 100 epochs with no early stopping, saving the best checkpoint by validation Dice and also the final epoch.
- Use Dice loss as in the paper, optionally combined with binary cross-entropy for steadier gradients; add a learning-rate scheduler and light augmentation (flips, rotations, brightness).
- Run inference on the full image using overlapping tiles blended together, rather than on isolated patches.

7.3 Attention U-Net

Plain skip connections pass every encoder feature to the decoder, including background clutter. An **attention gate** sits on each skip path. It takes the encoder feature x and a gating signal g from the deeper decoder layer, combines them through 1 by 1 convolutions, ReLU and a sigmoid, and produces an attention map between 0 and 1. The skip feature is multiplied by this map, so irrelevant regions are suppressed and vessel regions pass through. It is useful for thin, low-contrast vessels, but adds parameters and compute, and in the paper it did not beat Residual U-Net.

7.4 Side-by-side

	U-Net	Residual U-Net	Attention U-Net
Block	Two convolutions	Two convolutions + shortcut (F(x)+x)	Two convolutions, gated skip path
Main idea	Encoder-decoder with	Easier optimisation, better	Focus on vessel regions,

	U-Net	Residual U-Net	Attention U-Net
	skips	gradient flow	suppress background
Strength	Simple and fast	Best overall in the paper	Good on thin and faint vessels
Cost	Lowest	Slightly higher	Highest (attention gates)

8. Training setup

Setting	Paper	Rebuild
Patch size	256 x 256	512 x 512 (configurable, compare with 256)
Patches	3,000 vessel-containing pairs; 2,100 / 450 / 450 for train / validation / test (70:15:15)	Same ratio, regenerated at the new size; split by source image
Optimiser	Adam, learning rate 1e-4	Adam (or AdamW), 1e-4, with scheduler
Batch size	8	8 or lower depending on GPU memory
Loss	Dice loss	Dice (optionally + BCE)
Epochs	25	100, no early stopping
Hardware	NVIDIA Tesla T4	Colab or JupyterHub GPU
Inference time	About 0.01 to 0.02 s per 256 patch	Measure again at 512 px

Why larger images may help, and the cost. A vessel only a pixel or two wide at 256 pixels becomes two to four pixels wide at 512, so the network has more to work with. The price is about four times as many pixels per image, so more GPU memory, slower epochs and a smaller batch. More resolution does not guarantee higher accuracy, so keep the size as a setting and report the 256 versus 512 comparison.

Why 100 epochs. The paper's loss curves showed stable convergence at 25 epochs. With 100 epochs you get a fuller picture: you can show whether validation loss keeps improving or starts to rise (overfitting). Saving the best checkpoint by validation Dice protects you from overfitting late in the run.

9. Evaluation metrics, with a worked example

Segmentation is judged by comparing the predicted mask with the manual ground truth, pixel by pixel. Four counts matter: **TP** (vessel predicted as vessel), **FP** (background predicted as vessel), **FN** (vessel missed), **TN** (background correctly left alone).

Metric	Formula	What it tells you
Dice	2TP / (2TP + FP + FN)	Overlap between prediction and truth. Main metric for imbalanced masks
IoU	TP / (TP + FP + FN)	Overlap, stricter than Dice
Accuracy	(TP + TN) / all pixels	Share of pixels correct. Misleading, because background dominates
Sensitivity (recall)	TP / (TP + FN)	How many vessel pixels were found. Low means thin vessels are missed
Specificity	TN / (TN + FP)	How well background is left alone
AUC	Area under the ROC curve	Separation of vessel from background across all thresholds

Worked example. Take 10,000 pixels where 1,000 are truly vessel. A model finds TP = 800, misses FN = 200, wrongly marks FP = 200 background pixels, and correctly leaves TN = 8,800.

Metric	Calculation	Value
Dice	1600 / (1600 + 200 + 200)	0.800
IoU	800 / (800 + 200 + 200)	0.667
Accuracy	(800 + 8800) / 10000	0.960
Sensitivity	800 / 1000	0.800
Specificity	8800 / 9000	0.978

Now a useless model that predicts "background" for every pixel: TN = 9,000, FN = 1,000, TP = FP = 0. Its accuracy is **0.90** but its Dice and sensitivity are **0**. This is exactly why the paper says accuracy "can be misleading" and gives more weight to Dice and IoU, and why your 98.45% accuracy should always be quoted next to Dice 0.819.

10. Results reported in the paper

Model	Dice	IoU	Accuracy	Sensitivity	Specificity	AUC
U-Net	0.811	0.683	0.9703	0.7710	0.9883	0.94
Residual U-Net	0.819	0.696	0.9845	0.8327	0.9869	0.96
Attention U-Net	0.818	0.694	0.9696	0.7960	0.9817	0.95

Residual U-Net leads on Dice, IoU, accuracy, sensitivity and AUC. U-Net has slightly higher specificity. The differences in Dice are small (0.811 to 0.819), so with 133 source images the ranking should be stated with care, and a small gap is not strong evidence on its own. Compared with published methods on DRIVE and CHASE_DB1 (PA-Net, PAM-UNet, AttMSFCU-Net), the paper's Residual U-Net reports higher accuracy, but it is trained on the pooled four-dataset set rather than one dataset, so the comparison is not like-for-like.

Fill in after retraining at 512 px and 100 epochs:

Model	Dice	IoU	Accuracy	Sensitivity	Specificity	AUC
U-Net						
Residual U-Net						
Attention U-Net						

11. The EfficientNet-B0 classifier

11.1 Why this model sits after segmentation

The three segmentation networks only produce vessel masks. They cannot say anything about CVD, hypertension or DR. A classifier is needed to turn the retinal information into those predictions, and EfficientNet-B0 is the classifier already used in your project. So it comes **after** segmentation in the workflow, reading both the image and the mask. It is a shared second stage, not one of the three compared models.

11.2 Why EfficientNet-B0

EfficientNet scales network depth, width and input resolution together using one compound coefficient, which gives strong accuracy for few parameters. B0 is the smallest version (about 5 million parameters), so it trains and runs on modest hardware, which suits a hospital server without a large GPU. Its backbone ends in a 1280-dimensional feature vector.

11.3 Design in the rebuild

- **Input:** 4 channels: the preprocessed RGB image plus the vessel mask. The first convolution is widened from 3 to 4 input channels; the new channel's weights are initialised small or from the mean of the RGB weights so pretrained features are preserved.
- **Resolution:** 384 to 512 pixels, up from 224.
- **Heads:** same three: CVD (sigmoid), hypertension (sigmoid), DR (5-way softmax).
- **Loss:** weighted multi-task sum, as before; class weights or focal loss to handle rare DR grades and the 22% CVD prevalence.
- **Training:** patient-level split, 100 epochs, best checkpoint by validation AUC.
- **Threshold:** chosen by Youden's J on the validation set, then fixed for testing.

11.4 Youden's J, with an example

J = sensitivity + specificity - 1. The threshold that maximises J on validation data is the one that best balances catching true positives against false alarms. Example: at one threshold, sensitivity 0.90 and specificity 0.30 give J = 0.20; at another, sensitivity 0.70 and specificity 0.60 give J = 0.30, so the second is preferred. Your existing CVD threshold (0.3238) produced sensitivity 0.95 and specificity 0.05 on the test set, J = 0.007: the validation-tuned threshold did not carry over, which is a sign the model has almost no discriminative power on those images.

11.5 Evaluation of the classifier

Report ROC-AUC with 95% confidence intervals, PR-AUC (important when CVD is rare), sensitivity, specificity, precision, F1, Brier score and the confusion matrix, all at patient level by averaging image probabilities per patient. For DR use accuracy, weighted and macro F1, and a confusion matrix over the five grades. Macro F1 matters because it exposes a model that ignores rare grades.

Task	Existing model (mBRSET test)	Rebuild (fill in)
Hypertension ROC-AUC	0.458	
CVD ROC-AUC	0.563	
CVD PR-AUC	0.245	
DR accuracy / macro F1	0.735 / 0.188	

12. End-to-end system workflow [Rebuild]

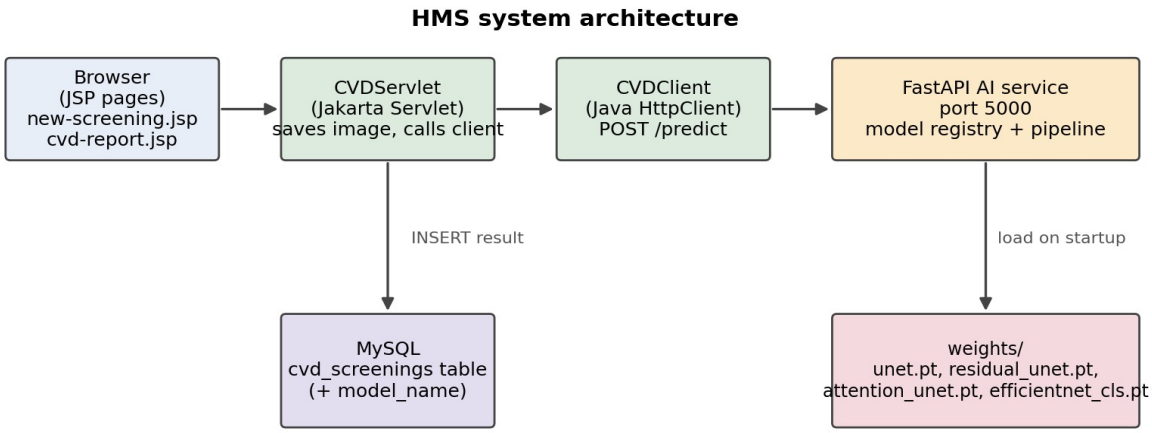


Figure 3. System architecture

12.1 A screening, step by step

1. The doctor opens **new-screening.jsp**, picks the patient, optionally the doctor and appointment, uploads a fundus image (or a bundled sample) and chooses a segmentation model from a dropdown.
2. **CVDServlet** reads the form, saves the image under `uploads/fundus/` with a unique name, and checks that the AI service is online through `CVDClient.isServiceAvailable()` (the `/health` endpoint).
3. **CVDClient** posts the image and the chosen model name to `/predict` on port 5000.
4. **FastAPI** resizes the image, applies green channel + CLAHE, looks up the chosen segmenter in the model registry, and produces the vessel mask using tiled inference.
5. The image and mask are stacked into 4 channels and passed to **EfficientNet-B0**, which returns CVD, hypertension and DR probabilities.
6. Thresholds turn probabilities into Positive/Negative; `compute_overall_assessment` produces the risk level and recommendation; the response also carries the mask overlay and a vessel density value.
7. **CVDClient** parses the JSON into a `CVDScreening` object; **CVDScreeningDAO** inserts it into MySQL, now including `model_name`.
8. The servlet redirects to **cvd-report.jsp**, which shows the original image, the preprocessed image, the mask overlay, the three results, the risk level and the disclaimer.

12.2 API design

Endpoint	Purpose
`GET /health`	Service and model-loaded check, used by the Java client
`GET /models`	Lists available segmentation models so the dropdown can be filled
`POST /predict` (file + `model`)	Runs the full pipeline with the chosen segmentation model
`POST /predict-all` (file)	Runs all three models and returns each mask plus an average
`POST /segment` (file + `model`)	Returns only the vessel mask, for demonstrations and for the paper figures

12.3 Database change

Add columns to `cvd_screenings`: `model_name` (which segmenter was used), `vessel_density`, and `mask_path`. Update `CVDScreening`, `CVDScreeningDAO.saveScreening`, and the report page. Replace the regex JSON parsing in `CVDClient` with a real JSON library, because the response will now contain nested objects.

13. Choosing the model at prediction time

Yes, the user can choose. The segmentation stage is plug-in based: a registry maps a name to an architecture class and a weights file, and the API accepts the name as a parameter.

Dropdown option	What happens	Use it for
Residual U-Net (default)	Segments with Residual U-Net, then classifies	Normal clinical-style use; best in the paper
U-Net	Segments with U-Net	Baseline and speed
Attention U-Net	Segments with Attention U-Net	Thin or faint vessels
Compare all 3	Runs all three, shows three masks side by side with their vessel density	Demonstrations and your paper's comparison

Caveat to understand. The EfficientNet-B0 classifier is trained on masks from one segmenter (the plan is Residual U-Net). If a user picks U-Net or Attention U-Net, the classifier sees masks that look slightly different from what it was trained on. Because the three models are close (Dice 0.811 to 0.819), the effect is probably small, but it should be measured and reported as an ablation. If you want exact consistency, train one classifier per segmenter, at roughly three times the classification training cost.

14. Folder structure [Rebuild]

```
cvd-ai-service/
  config/config.yaml            sizes, epochs=100, lr, batch size, thresholds
  data/                         not committed to git
    vessel/{DRIVE,STARE,CHASE_DB1,HRF}/
    brset/  mbrset/
    processed/{patches,generated_masks}/
  preprocessing/
    green_clahe.py  patching.py  preproc_metrics.py
  segmentation/
    models/unet.py  residual_unet.py  attention_unet.py
    dataset.py  losses.py  augment.py
    train_seg.py  evaluate_seg.py  infer_seg.py
  classification/
    efficientnet_multitask.py  train_cls.py  evaluate_cls.py
  serving/
    registry.py  pipeline.py  api_server.py
  weights/   unet.pt  residual_unet.pt  attention_unet.pt  efficientnet_cls.pt
  results/   curves, ROC, tables, mask overlays
src/main/java/com/hms/...       controller, dao, model, service (CVDClient)
src/main/webapp/...             JSP pages (model dropdown, mask view)
```

15. Build order

9. Set up data folders; implement and test `green_clahe.py` and the preprocessing metrics; reproduce Table I.
10. Implement patching at 512 px; split by source image.

11. Implement U-Net, Residual U-Net and Attention U-Net; run one epoch of each as a smoke test.
12. Train all three for 100 epochs; save curves, best and last checkpoints; evaluate and fill the results table.
13. Generate masks for BRSET and mBRSET with the best segmenter and check a sample by eye.
14. Train and evaluate the 4-channel EfficientNet-B0 at patient level.
15. Build the registry, pipeline and FastAPI endpoints; test with `curl`.
16. Update the Java client, DAO, database and JSPs; test the full flow.
17. Run ablations (256 vs 512 px, with vs without mask, classifier mask source); write up.

16. Limitations and honest caveats

- **Small segmentation set.** Only 133 images across four datasets; results may not generalise to new cameras or populations. The paper says this itself.
- **Domain gap.** Vessel models are trained on DRIVE/STARE/CHASE/HRF but applied to handheld-camera images in mBRSET; check generated masks visually.
- **Weak CVD signal.** The existing image-only CVD result is near chance. Adding masks may help but is not guaranteed; present CVD output as low-reliability unless the new results clearly show otherwise.
- **Labels are history, not future risk.** CVD labels record past vascular disease or heart attack, not a 5- or 10-year incident risk. HT has no numeric blood pressure, only a yes/no history.
- **Cohort.** Almost all patients are diabetic, so the system cannot predict diabetes and results may not apply to non-diabetic people.
- **Patch versus full image.** Patch metrics can look better than whole-image behaviour because patches are chosen to contain vessels.
- **Not a medical device.** For research and demonstration; predictions must not be used for diagnosis or treatment decisions. The reports carry this disclaimer.

17. Key facts and terms at a glance

Item	Value / meaning
Segmentation models	U-Net, Residual U-Net (best in paper), Attention U-Net
Classifier	EfficientNet-B0, 3 heads (CVD, hypertension, DR), 4-channel input in the rebuild
Preprocessing	Green channel then CLAHE; entropy 3.65, CII 0.264, EBCM 24.09
Vessel data	DRIVE 20 + STARE 40 + CHASE_DB1 28 + HRF 45 = 133 images; 3,000 patches; 2,100 / 450 / 450
Paper training	25 epochs, Adam 1e-4, batch 8, Dice loss, Tesla T4, 256 px patches
Rebuild training	100 epochs, 512 px, no early stopping, best checkpoint by validation metric
Paper best result	Residual U-Net: Dice 0.819, IoU 0.696, accuracy 98.45%, sensitivity 83.27%, specificity 98.69%, AUC 0.96
Existing classifier weakness	CVD AUC 0.563, HT AUC 0.458, DR macro F1 0.188 on mBRSET test
Thresholds (existing)	CVD 0.3238, HT 0.5165, DR confidence: top >= 0.50 and margin >= 0.15
Loss (existing classifier)	BCE(HT) + 2 x BCE(CVD) + CE(DR)
Service ports and files	FastAPI on 5000; weights in `weights/`; thresholds in `config`
Skip connection	Copies encoder features to the decoder to recover detail
Residual connection	Adds the block input to its output, F(x) + x

Item	Value / meaning
Attention gate	Learns a 0-to-1 map that suppresses irrelevant skip features
CLAHE	Contrast Limited Adaptive Histogram Equalisation, local contrast boost with a clip limit
Youden's J	Sensitivity + specificity - 1; used to pick the decision threshold
Patient-level split	All images of one patient stay in one subset, preventing leakage